



18 — Complex Formulas: \$derived.by

REACTIVITY LOGIC

Q. How do you calculate a derived value that requires multiple lines of logic, temporary variables, or loops?

ANSWER. Normally, a `$derived` value is just a single math equation. But if you need to filter a massive list of products based on five different search criteria, you need `if` statements and loops. You use `$derived.by` to run a massive, multi-line JavaScript function that unlocks the full power of the language while keeping the result perfectly cached.

Stats.svelte

```
<script>
  let numbers = $state([1, -5, 10, 42, -3]);

  // Use .by() when you need a full function block
  let positiveTotal = $derived.by(() => {
    let sum = 0;
    for (let num of numbers) {
      if (num > 0) sum += num;
    }
    return sum;
  });
</script>
```

\$derived

```
One line
only.
x * 2
```

\$derived.by

```
Full
function.
Loops
allowed.
If
statements.
Must
return.
```

① **Summary.** Sometimes a simple math expression isn't enough to calculate a derived value. This is powerful because `$derived.by` unlocks the full power of JavaScript inside your reactivity. You can write complex parsing or filtering algorithms that still auto-update efficiently.